

into more specific queries, in order to avoid long result lists. For instance, a query specifying an author and a conference returns a list of queries that further indicate all the publication years for the given author and conference. Although we do not expect index hierarchies to be very deep in practice, this scheme allows us to observe the effect of hierarchy depth.

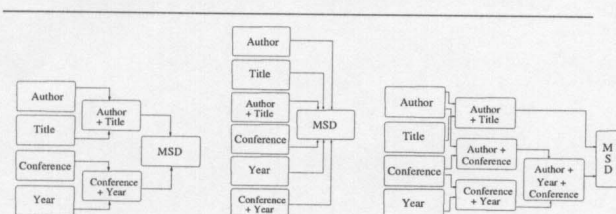


Figure 6. Indexing schemes: Simple (Left), Flat (Center) and Complex (Right).

The *simple* indexing scheme is the most space-efficient of the three, requiring 152 MB of extra storage in the system for the full DBLP article collection. The *complex* scheme requires 25% more space, and the *flat* scheme a 37% increase, being the most space-consuming. By comparison, we have evaluated the cost of storing the actual article files in the P2P infrastructure. Based on an average file size of 250 KB (estimated from the size of the articles in our own private digital library, counting more than 7,000 files in Postscript and Adobe PDF format), 29.1 GB are required to store all the articles in the DBLP archive. In the worst case, the indexes require an additional storage capacity of 0.5%. For a P2P network already storing the full article collection, index storage requirements are clearly negligible.

User Model. The mean number of interactions needed to reach an article gives a measure of the efficiency of a given indexing scheme. This can be computed as the average length of all possible query chains a user can follow to find an article. This is, however, only valid in a scenario where all kinds of queries are used with equal probability, and all articles are equally requested; such assumptions are not realistic.

To model realistic user behavior, we built a query generator that reflects which information is used to build queries (query structure), and which data is requested (data popularity).

Realistic query structure model: To generate realistic queries, we have used the information gathered from the query logs of the BibFinder and NetBib sites. For both of these archives, results from queries are always a list of matching articles. Any refinement made by a user (possibly overwhelmed by a huge list of results) is independent from the previous query. We may

then consider all queries as independent from each other. Both logs agree on the fact that queries are mainly made using author names, with conference and publication date as second and third criteria. We have therefore mapped our experimental query model directly to the structure and frequency of the queries in the BibFinder log, as shown in Figure 7 (Left).

Realistic popularity model: After modeling *how* articles are queried, we need to model *which* of them are most often requested. We have first observed author popularity, by counting the number of queries to each author in the BibFinder and NetBib log traces: the popularity of an author is defined as the probability of having a query with the “author” field being for that author. Similarly, we have computed the popularity of requested article titles in the BibFinder log. Finally, we have observed the probability that each of the top-10,000 articles in the CiteSeer database [20] gets cited; although this is a measure of how frequently a given article is cited, it can be considered as a clear indication of the article’s popularity.

The observed probability distribution are shown in Figure 7 (Center) (logarithmic scales, with articles ordered by decreasing rank of popularity). It appears that all probabilities follow roughly a power-law [21]. That is, the popularity of an article with rank i is $p_i = \frac{k}{i^\alpha}$, and $\sum p_i = 1$. A few articles appear in many queries, while most authors are seldom requested.

To model article popularity for our simulations, we have computed (using the minimum square method) from the plot of BibFinder’s author probabilities the line that best fits the distribution; switching to a linear scale, we obtain the power-law distribution describing the popularity of each article and the associated cumulative distribution function. In order to simplify the simulations, we have considered a limited collection of 10,000 articles. After adapting the parameters of the power-law distribution to match the finite population of articles, we obtain a complementary cumulative distribution function of $\bar{F}(i) = 1 - F(i) = 1 - 0.063 \cdot i^{0.3}$, where i is the ranking of the article. Numerical values are given for illustration purposes only: the behavior is defined by the *family* of the probability distribution function, i.e., power-law functions. The distribution function is plotted in Figure 7 (Right).

From the figure we can appreciate that, because of the skewed distribution, using only 10,000 articles does not change significantly the behavior of the model. The remaining articles from the original DBLP archive would be requested so seldom that we can effectively neglect their existence. When constructing the query workload for the simulation, we first choose an article according to the popularity distribution. Then, we select the structure of the query and assign the corresponding fields, according to the following probabilities: author only (with probability 0.6); title only (0.2); year only (0.1); both author and title (0.05); both author and year (0.05). The resulting query is subse-

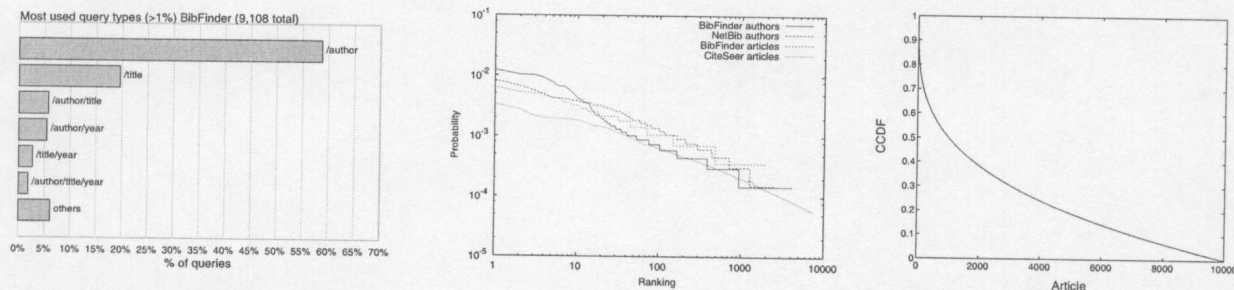


Figure 7. (Left) Distribution of the types of queries in BibFinder, (Center) popularity distribution for authors and titles in NetBib, BibFinder, and CiteSeer, (Right) Complementary cumulative distribution function of articles rankings.

quently used as input to locate the article selected initially.

Caching. As previously discussed, some articles are much more popular than others, and hence will be requested more frequently. By creating cache entries (shortcuts) for these articles, we can probabilistically improve lookup times and reduce the load on the system. We have observed the effectiveness of the adaptive caching mechanism described in Section 4, in which peers create cache entries along the lookup paths of successful queries. In the simulation, we study three different caching policies:

Multi-cache: Shortcuts are created on each node along the lookup path. The size of the cache is unbounded.

Single-cache: Shortcuts are created only on the first node that was contacted. The size of the cache is unbounded.

LRU: Similar to the single-cache policy, but only a limited number of shortcuts can be stored on each node. When the cache is full, a least-recently used (LRU) replacement algorithm is used.

Simulation. Simulating P2P networks of different sizes is of no use for our experiments. The number of nodes can affect the DHT lookup latency, and the number of keys stored per node, but does not impact the effectiveness of our indexing techniques. Any optimization of the underlying P2P network to reduce lookup latency will improve the response time when searching through the indexes, but these are completely independent issues (layered protocols). Our experiments simulate a P2P network of 500 nodes, on top of which a distributed bibliographic database storing 10,000 articles is implemented. Each simulation consists of sequentially feeding the indexing network with 50,000 queries from our query generator. The studied parameters are: number of user-system interactions required to find data, traffic generated, efficiency of shortcuts, and storage dedicated to caching. In all the figures of this section, *S*, *F*, and *C* stand for *simple*, *flat*, and *complex* indexing, respectively. The LRU replacement caching policy is tested for an al-

lowed maximum of 10, 20, and 30 cached keys per node.

User-system interactions: A user sends a query and obtains as result a list of more specific queries (covered by the original query). The user then selects one query from the results that matches the target article. This process iterates until the article is found. Isolated from other concerns, a user would like to experience a minimal number of iterations to locate the desired data. One should note, however, that the number of iterations is expected to increase when the user initiates the search with a generic query; there is clearly a trade-off between the amount of information initially known about the searched data, and the number of steps necessary to locate that data.

Simulation results are shown in Figure 8 (Left). The *flat* indexing scheme, which creates the shortest query chains (see Figure 6), also requires the fewest interactions to locate data. Caching further reduce the number of lookup steps, which becomes smaller as the cache capacity increases. The multi-cache policy is not shown here because it presents the same characteristics as the single-cache policy. Due to the rather simple representation of data, index chains are in general short. More complex data representations would need longer index chains, where the effect of shortcuts is more important.

Generated traffic: Like in any other network, it is desirable to generate as little traffic as possible to avoid network congestion and save system resources. Another more subtle interpretation of the traffic generated is the number of responses that a user receives for a query, i.e., the size of the result sets (traffic is mainly driven by responses, which usually outnumber a single query). The more traffic, the more responses a user gets for a query. Large results sets increase the burden of the user because responses are less relevant and more post-processing is required. Note that, since our search process is completely deterministic, more results do not imply that more information is available (as could be the case of an Internet search engine such as Google [22]), but a *less precise answer*. The query and response traffic measured during the simulations

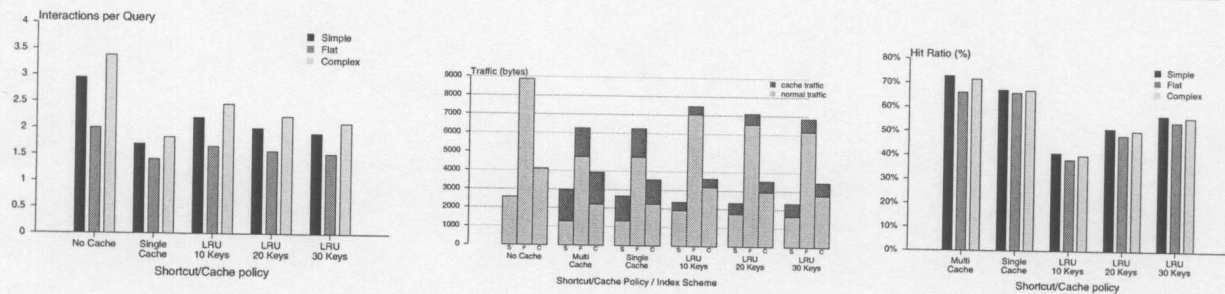


Figure 8. (Left) Average number of interactions required to find data, (Center) average network traffic (bytes) generated per query, (Right) cache efficiency: distributed hit ratio.

is shown in Figure 8 (Center).

Cache traffic, resulting from the creation of cache entries after successful lookups, is shown in dark gray. It appears clearly that the *flat* indexing scheme generates much more traffic than any other. In fact, it does not allow for any indirection, and each query receives directly the descriptors of *all* articles that match the query. We observe that the utilization of the cache saves network bandwidth. Unsurprisingly, larger cache sizes yield more cache traffic and less overall traffic transmitted over the network.

Cache efficiency: We have observed the effectiveness of the adaptive distributed cache by measuring its hit ratio, that is, the fraction of requests that do not need to go through a full search process because the relevant data is already available in the cache.

Figure 8 (Right) shows the results for the different policies tested. It is interesting to note that the multi-cache policy is only marginally more efficient than the single-cache policy. Most cache hits occur in the first node of the chain (86% for the *simple* scheme, 99.9% for *flat*, and 84% for *complex*). Indeed, in our user model queries are usually very simple and broad, thus directing the user to the beginning of an index chain.

Cache storage: The storage dedicated to cached keys should remain relatively small, while allowing for an efficient distributed cache. The number of regular keys stored on each node depends on the indexing scheme. After feeding the 50,000 queries, we observed an average of 155 keys per node for *simple*, 195 for *flat*, and 180 for *complex*.

As expected, the single-cache policy is approximately twice as space-efficient as the multi-cache policy, which creates more cache entries. The *flat* indexing scheme is unaffected because its index chains are so short that they only allow for a key to be cached in the first node.

We also observed the maximum number of cache entries stored across all nodes. For the multi-cache policy, we find up to 345 cached keys in a node with the *simple* indexing. For the *flat* and *complex* indexing, the maxima are 253 and 413 keys, respectively. The single-cache

policy uses a maximum of 253 cached keys, regardless of the indexing scheme used. For the limited cache size policies, the maxima are obviously the cache capacities (10, 20, and 30). 72% of the caches were full after feeding the 50,000 queries with the LRU10 policy, 51.2% with LRU20, and 37.6% with LRU30, regardless of the indexing scheme. 4.4% of the caches were not used at all. As the cache management directly depends on the query workload, cache utilization is not uniform.

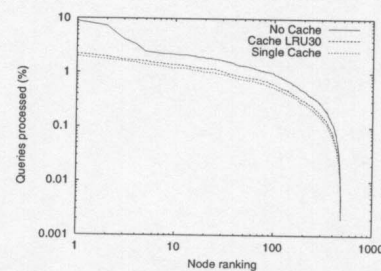


Figure 9. Percentage of queries processed by each node in the network.

Hot-spots: As just discussed, the user query patterns leads to an unbalanced utilization of the distributed cache. In Figure 9, we show the percentage of the 50,000 queries issued during the simulation that accessed each node (for clarity, we only show the results for the *simple* indexing scheme). Note that they sum to more than 100% because each original query may generate other queries during the iterative lookup process. We observe that the busiest node is affected by almost 1 out of every 10 queries. Caching slightly improves the situation for the most severely stressed nodes. We expect the load to be better balanced with larger content and query workloads. Note also that any optimization of the underlying P2P DHT substrate for hot-spot avoidance (e.g., using replication) will apply to index accesses as well.

Locating non-indexed data: As previously mentioned, it may happen that a user issues a query using a combination of fields (author, year, etc.) that has not been indexed. In that situation, the system can first generalize the original query to find a matching index entry, and then specialize it by following indexes until the target data is located. In our experiments, this event happened approximately 2,500 times for all indexing schemes when no cache is used. When an error is encountered, one extra interaction is generally necessary to find a suitable index (two interactions in a few rare cases).

	Simple	Flat	Complex
No cache	2,502	2,507	2,506
LRU30	810	874	838
Single-cache	563	600	581

Table 1. Number of queries to non-indexed data.

Table 1 shows the number of accesses to non-indexed data, i.e., recoverable errors, as a function of the cache policy deployed in the system. We can observe that the cache reduces the number of errors, because an index entry is created automatically after the first lookup; subsequent queries from other users can locate the data using the cache entry, and hence do not experience an error. Our indexing techniques, together with the adaptive distributed cache, are thus very flexible: although the indexing scheme is generally chosen at deployment time, the system can still adapt to the user querying habits by creating shortcuts dynamically.

6. Final Notes

A major limitation of DHT peer-to-peer system is that they only support exact-match lookups: one needs to know the exact key of a data item to locate the node responsible for storing that item. Since peer-to-peer users tend to submit broad queries to look up data items, DHT peer-to-peer systems need to be augmented with mechanisms for locating data using incomplete information.

In this paper, we have proposed techniques for indexing the data stored in the peer-to-peer network. Indexes are distributed across the nodes of the network and contain key-to-key (or query-to-query) mappings. Given a broad query, a user can look up the more specific queries that match its original query; the DHT can be recursively queried until the user finds the desired data items. As they can be layered on top of an arbitrary P2P DHT substrate, our indexing techniques directly benefit from any mechanisms implemented in the DHT to deal with failures or hot-spot avoidance. We have performed a comprehensive evaluation to demonstrate the effectiveness of data indexing on a distributed P2P bibliographic database, using different strategies and realistic user query workloads. Finally, we have proposed an adaptive caching mechanism that has proved to improve performance of accesses to popular content.

Although our data indexing techniques permit to look up data based on incomplete information, they still depend on the exact matching facilities of the underlying DHT. "Fuzzy" matching techniques offer interesting research perspectives for dealing with misspelled data descriptors or queries. Misspellings can also often be taken care of by validating descriptors and queries against databases that store known file descriptors, such as CDDb [23] for music files.

References

- [1] I. Stoica, R. Morris, D. Karger, M. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in *Proceeding of SIGCOMM*, 2001.
- [2] S. Ratnasamy, M. Handley, R. Karp, and S. Shenker, "A scalable content-addressable network," in *Proceeding of SIGCOMM*, 2001.
- [3] A. Rowstron and P. Druschel, "Pastry: Scalable, distributed object location and routing for large-scale peer-to-peer systems," in *Proceedings of Middleware*, Nov 2001.
- [4] B.Y. Zhao, J. Kubiatowicz, and A.D. Joseph, "Tapestry: An infrastructure for fault-tolerant wide-area location and routing," Tech. Rep. UCB/CSD-01-1141, University of California, Berkeley, Apr 2001.
- [5] M. Balazinska, H. Balakrishnan, and D. Karger, "INS/Twine: A scalable peer-to-peer architecture for intentional resource discovery," in *Proceedings of the International Conference on Pervasive Computing*, August 2002.
- [6] W. Adjie-Winoto, E. Schwartz, H. Balakrishnan, and J. Lilley, "The design and implementation of an intentional naming system," in *Proceedings of SOSP*, 1999.
- [7] M. Harren, J. Hellerstein, R. Huebsch, B. Loo, S. Shenker, and I. Stoica, "Complex queries in dht-based peer-to-peer networks," in *Proceedings of IPTPS*, Cambridge, USA, March 2002.
- [8] A. Gupta, D. Agrawal, and A. Abbadi, "Approximate range selection queries in peer-to-peer systems," Tech. Rep. UCSB/CSD-2002-23, University of California at Santa Barbara, 2002.
- [9] O.D. Sahin, A. Gupta, D. Agrawal, and A. Abbadi, "Query processing over peer-to-peer data sharing systems," Tech. Rep. UCSB/CSD-2002-28, University of California at Santa Barbara, 2002.
- [10] E. Cohen, A. Fiat, and H. Kaplan, "Associative search in peer to peer networks: Harnessing latent semantics," in *Proceedings of Infocom*, San Francisco, CA, April 2003.
- [11] "Gnutella," <http://gnutella.wego.com>.
- [12] J. Li, B.T. Loo, J. Hellerstein, F. Kaashoek, D. Karger, and R. Morris, "On the feasibility of peer-to-peer web indexing and search," in *Proceedings of IPTPS*, Berkeley, CA, February 2003.
- [13] F. Dabek, M.F. Kaashoek, David Karger, Robert Morris, and Ion Stoica, "Wide-area cooperative storage with cfs," in *Proceedings of SOSP*, Oct. 2001.
- [14] A. Rowstron and P. Druschel, "Storage management and caching in past, a large-scale, persistent peer-to-peer storage utility," in *Proceedings of SOSP*, Oct. 2001.
- [15] W3C, "Extensible Markup Language (XML) 1.0," <http://www.w3.org/TR/REC-xml>, Oct. 2000.
- [16] "DBLP," <http://dblp.uni-trier.de>
- [17] W3C, "XML Path Language (XPath) 1.0," <http://www.w3.org/TR/xpath>, Nov. 1999.
- [18] "Bibfinder," <http://kilimanjaro.eas.asu.edu>
- [19] "Netbib," <http://edas.info/S.cgi?search=1>
- [20] "Citeseer," <http://citeseer.nj.nec.com/cs>
- [21] G.K. Zipf, *Human Behavior and the principle of least effort*, Addison-Wesley Press, Cambridge (MA), 1949.
- [22] "Google," <http://www.google.com>
- [23] "CDDb," <http://www.cddb.org>